

- ① Define the algorithm, show its implementation in python
- ② Define its continuous time version, define the Boltzmann-Gibbs distribution and show a numerical example
- ③ Define its kinetic formulation
 - ① Explicitly wrote $\mathcal{L}_{\mathcal{F}}$ operator and compared it to the classical $Q(f,f)$ operator
 - ② found the equilibrium density leveraging $\mathcal{L}_{\mathcal{F},t}$ integral definition explicitly computed and compared it to the classical Maxwellian
 - ③ found the collision invariants leveraging the weak formulation and comment on the similarities
- ④ Show convergence of the algorithm through its kinetic formulation and an analogous of H theorem

Classical Simulated Annealing 1

We consider the optimization problem: $x^* \in \operatorname{argmin}_{x \in \mathbb{R}^d} \mathcal{F}(x)$.

- Initialization:

$$X^{(0)} \in \mathbb{R}^d, \quad T^{(0)} > 0$$

- Update:

$$\tilde{X}^{(n+1)} = X^{(n)} + \sigma^{(n)} \xi, \quad \xi \in \mathbb{R}^d, \quad \sigma^{(n)} = \sigma(T^{(n)})$$

- Acceptance rule:

$$X^{(n+1)} = \begin{cases} \tilde{X}^{(n+1)}, & \mathcal{F}(\tilde{X}^{(n+1)}) \leq \mathcal{F}(X^{(n)}), \\ \tilde{X}^{(n+1)} \text{ with probability } p, & \mathcal{F}(\tilde{X}^{(n+1)}) > \mathcal{F}(X^{(n)}), \\ X^{(n)} \text{ with probability } 1 - p, & \mathcal{F}(\tilde{X}^{(n+1)}) > \mathcal{F}(X^{(n)}), \end{cases}$$

with

$$p = \exp \left(- \frac{\mathcal{F}(\tilde{X}^{(n+1)}) - \mathcal{F}(X^{(n)})}{T^{(n)}} \right)$$

Classical Simulated Annealing 2

The previous rule can be written more compactly by defining:

$$P_{\mathcal{F}}^{(n)} = \min \left\{ 1, \exp \left(- \frac{\mathcal{F}(\tilde{X}^{(n+1)}) - \mathcal{F}(X^{(n)})}{T^{(n)}} \right) \right\}$$

Let $\eta \sim \mathcal{U}(0, 1)$; notice if $\mathcal{F}(\tilde{X}^{(n+1)}) > \mathcal{F}(X^{(n)})$, $\mathbf{1}_{\{\eta < P_{\mathcal{F}}^{(n)}\}} = 1$ with probability $P_{\mathcal{F}}^{(n)}$. If $\mathcal{F}(\tilde{X}^{(n+1)}) < \mathcal{F}(X^{(n)})$, $\mathbf{1}_{\{\eta < P_{\mathcal{F}}^{(n)}\}} = 1$ $\mathbb{P}$ -a.s. So we can write:

$$X^{(n+1)} = \mathbf{1}_{\{\eta < P_{\mathcal{F}}^{(n)}\}} \tilde{X}^{(n+1)} + \left(1 - \mathbf{1}_{\{\eta < P_{\mathcal{F}}^{(n)}\}} \right) X^{(n)}$$

Simulated Annealing Implementation

We work in the one-dimensional case, so $X^{(n)} \in \mathbb{R}$, and we take

$$\tilde{X}^{(n+1)} = X^{(n)} + \sqrt{T^{(n)}} \xi, \quad \xi \sim \mathcal{N}(0, 1)$$

The cooling schedule is: $T^{(n)} = \frac{T_0}{\log(n+2)}$

```
def simulated_annealing( fun: Callable[[float], float], x0: float, t0: float,
n_iter: int = 20000, seed: int | None = 762) -> tuple[float, float]:
    x,value = x0, float(fun(x))
    best_x,best_value = x,value

    for n in range(n_iter):
        temp = t0 / np.log(n + 2)
        sigma = np.sqrt(temp)
        x_prop = x + sigma * rng.normal()
        value_prop = float(fun(x_prop))
        x, value = acceptance_rule(x, value, x_prop, value_prop, temp, rng)

        if value < best_value:
            best_x = x
            best_value = value

    return best_x, best_value
```

Simulated Annealing Implementation - Results

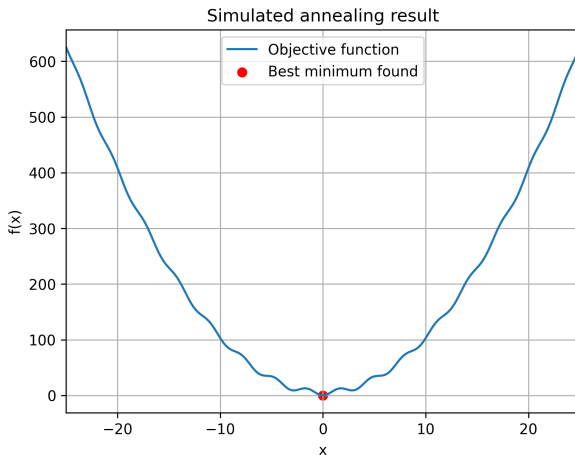


Figure: Cost function and best point found after 2e5 iterations

Simulated Annealing in Continuous Time

The algorithm position at time t in continuous time is described by:

$$dX_t = -\nabla \mathcal{F}(X_t) dt + \sqrt{2T} dB_t$$

That means a path of the solution of this SDE is one realization of the algorithm. Calling $f(x, t)$ the pdf of the random variable X_t we have:

$$\mathbb{P}(X_t \in A) = \int_A f(x, t) dx$$

More importantly its time evolution is described by the Fokker-Planck equation:

$$\partial_t f(x, t) = \nabla_x \cdot (f(x, t) \nabla \mathcal{F}(x)) + T \Delta_x f(x, t)$$

For fixed x , this describes how the probability density changes in time. Roughly speaking, it allows us to understand whether probability mass around x is increasing or decreasing.

Boltzmann-Gibbs Distribution

For a fixed temperature $T > 0$, we define

$$f_{\mathcal{F},T}^{\infty}(x) = \lim_{t \rightarrow \infty} f(x, t) = \frac{1}{Z_T} \exp\left(-\frac{\mathcal{F}(x)}{T}\right)$$

which is the stationary solution of the Fokker-Planck equation.

Since T is fixed, this does not represent convergence of the algorithm to the global minimizer, but rather its long-term distribution at temperature T . If the global minimizer x^* is unique, then

$$f_{\mathcal{F},T}^{\infty} \rightarrow \delta_{x^*}, \quad \text{as } T \rightarrow 0$$

The important point is that the stationary density can be derived explicitly: it is the Boltzmann-Gibbs distribution.

Boltzmann-Gibbs Distribution: Numerical

We need to repeat the simulated annealing for a fixed T (hence fixed σ), stop after $N \gg 0$ iteration and repeat this process M times to get an empirical long term distribution. Then we can compare it with the Boltzmann-Gibbs one, where the Z_T term is numerically approximated.

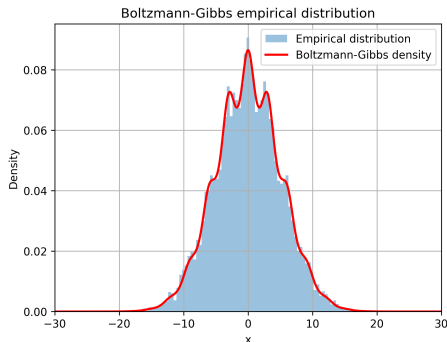


Figure: Empirical distribution of the final states $X_i^{(1000)}$ from 10000 realizations

Pareschi moves to the kinetic description of the problem in order to:

- Avoid the gradient in the X_t dynamics, which doesn't appear in the original algorithm
- Mainly: to use statistical mechanics tools to show convergence

Let $f(t, x)$ be the pdf of X_t with $\int_{\mathbb{R}^d} f(x, t) dx = 1 \forall t$. Then:

$$\frac{\partial f(x, t)}{\partial t} = \langle B_{\mathcal{F}}(x' \rightarrow x)f(x', t) - B_{\mathcal{F}}(x \rightarrow x')f(x, t) \rangle =: (\mathcal{L}_{\mathcal{F}}f)(x, t)$$

Recall classical Boltzmann equation:

$$\frac{\partial f(x, \xi, t)}{\partial t} + \frac{\partial f(x, \xi, t)}{\partial x} = Q(f, f)$$

The two equations share the same gain-loss structure; in simulated annealing $\mathcal{L}_{\mathcal{F}}$ is linear in f , while the Boltzmann operator $Q(f, f)$ is nonlinear.

$$\underbrace{\frac{\partial f}{\partial t} + \xi \cdot \nabla_x f = Q(f, f)}_{\text{Boltzmann}} \qquad \underbrace{\frac{\partial f(x, t)}{\partial t} = \mathcal{L}_{\mathcal{F}}(f)(x, t)}_{\text{SA}}$$

Recall that in classical Boltzmann the term $\nabla_x f$ comes from the fact that $\frac{d}{dt}f(x(t), \xi, t) = \frac{\partial}{\partial t}f + \frac{dx}{dt}\nabla_x f$. In the SA description, there is no transport term since the state evolution is always random. In Boltzmann we can imagine the model is deterministic + stochastic jump (after collision).

The two equations share the same gain-loss structure since in both cases the change in probability is described as the probability mass entering that state minus the probability mass leaving it. In SA the gain at x is the probability of being at x' , proposing and accepting a move to x .

The $\mathcal{L}_{\mathcal{F}}$ Operator

$$(\mathcal{L}_{\mathcal{F}}f)(x, t) = \langle B_{\mathcal{F}}(x' \rightarrow x)f(x', t) - B_{\mathcal{F}}(x \rightarrow x')f(x, t) \rangle$$

where:

- $B_{\mathcal{F}}(x \rightarrow x') = \min \left\{ 1, \exp \left(- \frac{\mathcal{F}(x') - \mathcal{F}(x)}{T(t)} \right) \right\}$ is the probability of accepting a move from $x' \rightarrow x$
- $\langle g \rangle = \mathbb{E}[g(\xi)] = \int_{\mathbb{R}^d} g(z) p_{\xi}(z) dz$

Notice that in our context we can write $x' = x + \sigma(t)\xi$, so the important random variable is ξ with symmetric pdf p_{ξ} . For example we can take $\xi \sim N$. In the project I have done all the explicit computations to write the operator in its integral form (see the long document). In the end:

$$(\mathcal{L}_{\mathcal{F},t}f)(x, t) = \int_{\mathbb{R}^d} B_{\mathcal{F}}(y \rightarrow x)f(y, t)p_Z(x - y, t) dy - f(x, t) \int_{\mathbb{R}^d} B_{\mathcal{F}}(x \rightarrow y)p_Z(y - x, t) dy$$

The $\mathcal{J}_{\mathcal{F},t}$ Operator 1

We define

$$\mathcal{J}_{\mathcal{F},t} = \mathcal{L}_{\mathcal{F},t} + I \quad \implies \quad \mathcal{J}_{\mathcal{F},t}(f) = \mathcal{L}_{\mathcal{F},t}(f) + f$$

Using the integral representation of $\mathcal{L}_{\mathcal{F},t}$:

$$(\mathcal{J}_{\mathcal{F},t}f)(x, t) = \underbrace{\int_{\mathbb{R}^d} B_{\mathcal{F}}(y \rightarrow x) f(y, t) p_Z(x - y, t) dy}_{\text{accepted moves into } x} + \underbrace{\left[1 - \int_{\mathbb{R}^d} B_{\mathcal{F}}(x \rightarrow y) p_Z(y - x, t) dy \right] f(x, t)}_{\text{rejected moves remaining at } x}$$

Probabilistic interpretation

$\mathcal{J}_{\mathcal{F},t}$ maps the current probability density into the density obtained after one complete Metropolis proposal–acceptance/rejection update.

The $\mathcal{J}_{\mathcal{F},t}$ Operator 2

The operator $\mathcal{J}_{\mathcal{F},t}$ preserves the probabilistic structure:

$$f \geq 0 \quad \implies \quad \mathcal{J}_{\mathcal{F},t}(f) \geq 0$$

$$\int_{\mathbb{R}^d} \mathcal{J}_{\mathcal{F},t}(f)(x) dx = \int_{\mathbb{R}^d} f(x) dx$$

Hence, if f is a probability density, then $\mathcal{J}_{\mathcal{F},t}(f)$ is again a probability density.

Equilibrium as a fixed point

Since $\mathcal{J}_{\mathcal{F},t} = \mathcal{L}_{\mathcal{F},t} + I$:

$$\mathcal{L}_{\mathcal{F},t}(f) = 0 \quad \iff \quad \mathcal{J}_{\mathcal{F},t}(f) = f$$

For fixed T , the Boltzmann-Gibbs distribution satisfies this condition (see full document). This is the analogue of the Maxwellian equilibrium $Q(f, f) = 0$ in the classical Boltzmann equation.

Weak formulation 1

Let $\varphi : \mathbb{R}^d \rightarrow \mathbb{R}$ be a test function. Since

$$\int_{\mathbb{R}^d} \varphi(x) f(x, t) dx = \mathbb{E}[\varphi(X_t)],$$

the weak formulation describes the evolution of observables of the process:

$$\frac{d}{dt} \int_{\mathbb{R}^d} f(x, t) \varphi(x) dx = \left\langle \int_{\mathbb{R}^d} B_{\mathcal{F}}(x \rightarrow x') (\varphi(x') - \varphi(x)) f(x, t) dx \right\rangle$$

Take $\varphi(x) = 1$; we immediately get:

$$\frac{d}{dt} \int_{\mathbb{R}^d} f(x, t) dx = 0,$$

so total probability is conserved. Which is a nice consistency check of our computations so far. (It would be problematic if this, for example, was ≥ 0)

Weak formulation 2

In classical Boltzmann we have followed a similar path without naming it "weak formulation". We took $\varphi(\xi)$ test function and

$$\int_{\mathbb{R}^3} Q(f, f)(\xi) \varphi(\xi) = 0 \iff \varphi(\xi_1) + \varphi(\xi_2) = \varphi(\xi'_1) + \varphi(\xi'_2)$$

precisely by "symmetrizing" the integral on the lhs. We then found that $\varphi(x) = 1, \varphi(x) = x, \varphi(x) = x^2$ are collision invariant. The same doesn't hold in the Kinetic formulation of simulated annealing.

$$\frac{d}{dt} \int_{\mathbb{R}^d} f(x, t) \varphi(x) dx = \frac{d}{dt} \mathbb{E}[\varphi(X_t)]$$

If $\varphi(x) = x$ was a collision invariant, it would mean that the mean is conserved over time which is clearly not the case for simulated annealing: we want the mean to move according to the algorithm.

H-Theorem & Entropy

In classical Boltzmann:

$$H(t) = \int_{\mathbb{R}^3} f(x, \xi, t) \log f(x, \xi, t) d\xi, \quad \frac{dH}{dt} \leq 0$$

We can define the relative entropy with respect to the equilibrium Maxwellian M as

$$H(f \mid M) = \int_{\mathbb{R}^3} f(\xi, t) \log \left(\frac{f(\xi, t)}{M(\xi)} \right) d\xi$$

and provides a measure of how different the current distribution f is from the equilibrium distribution M (this last part not seen during lecture, but it's precisely what we mention).

Simulated Annealing Equilibrium

In our case, define the relative entropy with respect to the Boltzmann–Gibbs distribution:

$$H(f \mid f_{\mathcal{F}}^{\infty}) = \int_{\mathbb{R}^d} f(x, t) \log \left(\frac{f(x, t)}{f_{\mathcal{F}}^{\infty}(x)} \right) dx$$

Differentiating in time we get:

$$\begin{aligned} \frac{d}{dt} H(f \mid f_{\mathcal{F}}^{\infty}) &= \int_{\mathbb{R}^d} \partial_t f \log \left(\frac{f}{f_{\mathcal{F}}^{\infty}} \right) dx \\ &= \int_{\mathbb{R}^d} \mathcal{L}_{\mathcal{F}}(f) \log \left(\frac{f}{f_{\mathcal{F}}^{\infty}} \right) dx \end{aligned}$$

It is possible to show (see full document for computations):

$$\frac{d}{dt} H(f \mid f_{\mathcal{F}}^{\infty}) = -I_{\mathcal{F}}[f] \leq 0.$$

Thus, the relative entropy is decreasing. The remaining question is how fast it converges to zero. With further computations, we can show:

$$\|f(\cdot, t) - f_{\mathcal{F}}^{\infty}\|_{L^1(\mathbb{R}^d)} \longrightarrow 0 \quad \text{as } t \rightarrow \infty.$$

Equilibrium - Numerical Experiment

Fixed-temperature relaxation ($T = 5.0$)

